

Name: Nargiz Mammadzada

ID: 8966

Microprocessor Systems

Lab Report 2

Task 6: Using register-level programming and parallel I/O, configure the microcontroller's hardware timer to drive a single-digit 7-segment display. The display should count from 0 to 9, updating once every 0.5 seconds. A push button connected to an external interrupt should pause the counter when pressed and resume it when pressed again. After the display reaches 9, it resets. Questions to think about:

- How can you be confident that it is counting in precise 0.5-second increments? How did you calculate this?
- Does the counter resume exactly where it left off?

GitHub Solution: <https://github.com/catponyz/Microprocessor-Systems/tree/main/task%206>

Demo Video: <https://youtu.be/4C4q3dPMt78>

Hardware Setup

For this task, the hardware was an Arduino Uno with ATmega328p, a single digit 7 segment display (common cathode type), 330 ohm current limiting resistors for the digits, a push button, breadboard, jumper wires, and an oscilloscope. The segments a–f were connected to PB0–PB5 and the segment g was connected to PC0. The button was wired to PD2, the external interrupt pin INT0 which is digital pin 2 of Arduino. Pause-state evidence signal: PD6, Timing evidence signal: PD7. The timing waveform was measured by the oscilloscope, as was the pause/resume behavior.

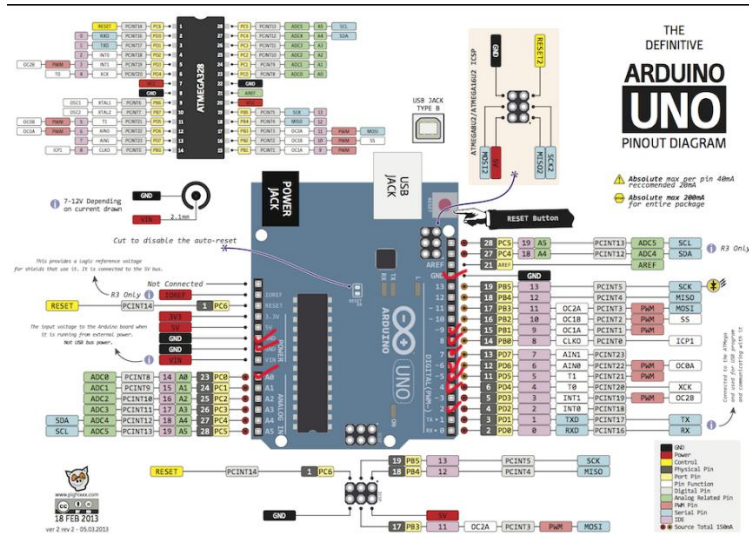


Figure 1. Arduino Uno pinout for Task 6

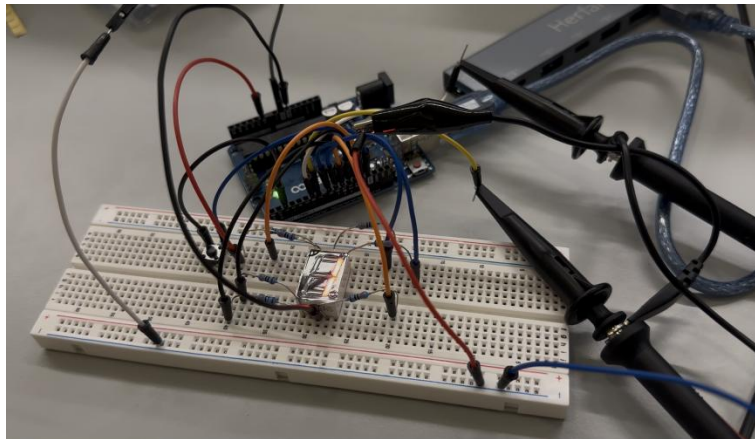


Figure 2. Hardware configuration for Task 6

Description

This task is programmed at the register level, with parallel I/O, Timer1 and an external interrupt controlling a 7-segment display. In general, a 7-segment display will display digits by activating certain LED segments and deactivating the rest. The `seg[]` array in this code contains the pattern for each digit (0-9). Each bit of the pattern corresponds to one segment: bit 0 is the segment a, bit 1 is the segment b, etc, bit 6 is the segment g.

`show_digit()` is a function that is used in the code to send the correct bit pattern to the display. The lower six bits are written to `PORTB`, corresponding to segments a-f. A segment g is connected independently to `PORTC0` and bit 6 of the digit pattern is shifted and written to `PORTC0`.

The digit is changed every 0.5 seconds with Timer1. Typically, the hardware timer is an independent counter of the clock. If the timer reaches a particular value, it may trigger an interrupt. The Timer1 is set up to a CTC mode, therefore it resets automatically as it reaches OCR1A. The TIMER1_COMPA_vect is called by every compare match. If the counter is not paused, the digit is incremented by $(\text{digit} + 1) \% 10$, which means that once it reaches 9, it resets to 0. This is then displayed.

The button is connected to external interrupt INT0 (on PD2). For the most part, an external interrupt will enable the microcontroller to respond instantly to any change in the state of the button as opposed to continually checking the button in the main loop. PD2 is pulled low when button is pressed, which causes a falling edge interrupt in the code. The interrupt toggles the paused variable. If the display is stopped, the digit remains displayed. If it is resumed, the sequence continues from the same digit.

Register / datasheet justification

The DDRB controls the direction of the pins of the Port B. $\text{DDRB} |= 0x3F$ makes the output pins PB0 to PB5 (also known as D4 to D9) the output display segments a to f. PORTB then drives those segment pins HIGH or LOW.

$\text{DDRC} |= (1 << \text{PC0})$ makes the PC0 line output for segment g. Since segment g was not connected to Port B, it will be updated separately, by PORTC0.

$\text{DDRD} |= (1 << \text{PD6})$ and $\text{DDRD} |= (1 << \text{PD7})$ set PD6 and PD7 as output evidence pins. The pause state of PD6 is displayed as HIGH or LOW. LOW indicates that PD6 is running, HIGH is paused. PD7 toggles every successful digit update (which is 0.5 second), and is used to prove the timer interval.

$\text{DDRD} \&= \sim(1 << \text{PD2})$ makes PD2 an input for the button. The internal pull-up resistor is enabled by using the following code: $\text{PORTD} |= (1 << \text{PD2})$; This means that the input is normally HIGH. Pressing the button will set PD2 to LOW.

$\text{EICRA} = (1 << \text{ISC01})$ sets INT0 to trigger on a falling edge, as PD2 is pulled low during a push. $\text{EIMSK} = (1 << \text{INT0})$ - sets the INT0 external interrupt.

12.2.1 EICRA – External Interrupt Control Register A

The external interrupt control register A contains control bits for interrupt sense control.

Bit	7	6	5	4	3	2	1	0	
(0x69)	–	–	–	–	ISC11	ISC10	ISC01	ISC00	EICRA
Read/Write	R	R	R	R	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 12-2. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

Figure 3. EICRA register and its ISC01/ISC00 bits

12.2.2 EIMSK – External Interrupt Mask Register

Bit	7	6	5	4	3	2	1	0	
0x1D (0x3D)	–	–	–	–	–	–	INT1	INT0	EIMSK
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

• Bit 0 – INT0: External Interrupt Request 0 Enable

When the INT0 bit is set (one) and the I-bit in the status register (SREG) is set (one), the external pin interrupt is enabled. The interrupt sense control0 bits 1/0 (ISC01 and ISC00) in the external interrupt control register A (EICRA) define whether the external interrupt is activated on rising and/or falling edge of the INT0 pin or level sensed. Activity on the pin will cause an interrupt request even if INT0 is configured as an output. The corresponding interrupt of external interrupt request 0 is executed from the INT0 interrupt vector.

Figure 4. EIMSK register and its INT0 enable bit

Setting TCCR1A = 0 will turn waveform output modes off. TCCR1B = (1 << WGM12) | (1 << CS12) selects CTC mode and sets Timer1 prescaler to 256. OCR1A = 31249 is the compare value for 0.5 seconds. TIMSK1 = (1 << OCIE1A) enables the Timer1 Compare Match A interrupt. sei() enables global interrupts.

15.11.2 TCCR1B – Timer/Counter1 Control Register B

Bit	7	6	5	4	3	2	1	0	
(0x81)	ICNC1	ICES1	–	WGM13	WGM12	CS12	CS11	CS10	TCCR1B
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 15-6. Clock Select Bit Description

CS12	CS11	CS10	Description
0	0	0	No clock source (Timer/Counter stopped).
0	0	1	clk _{IO} /1 (no prescaling)
0	1	0	clk _{IO} /8 (from prescaler)
0	1	1	clk _{IO} /64 (from prescaler)
1	0	0	clk _{IO} /256 (from prescaler)
1	0	1	clk _{IO} /1024 (from prescaler)
1	1	0	External clock source on T1 pin. Clock on falling edge.
1	1	1	External clock source on T1 pin. Clock on rising edge.

Table 15-5. Waveform Generation Mode Bit Description⁽¹⁾

Mode	WGM13	WGM12 (CTC1)	WGM11 (PWM11)	WGM10 (PWM10)	Timer/Counter Mode of Operation	TOP	Update of OCR1x at	TOV1 Flag Set on
0	0	0	0	0	Normal	0xFFFF	Immediate	MAX
1	0	0	0	1	PWM, phase correct, 8-bit	0x00FF	TOP	BOTTOM
2	0	0	1	0	PWM, phase correct, 9-bit	0x01FF	TOP	BOTTOM
3	0	0	1	1	PWM, phase correct, 10-bit	0x03FF	TOP	BOTTOM
4	0	1	0	0	CTC	OCR1A	Immediate	MAX
5	0	1	0	1	Fast PWM, 8-bit	0x00FF	BOTTOM	TOP
6	0	1	1	0	Fast PWM, 9-bit	0x01FF	BOTTOM	TOP
7	0	1	1	1	Fast PWM, 10-bit	0x03FF	BOTTOM	TOP
8	1	0	0	0	PWM, phase and frequency correct	ICR1	BOTTOM	BOTTOM
9	1	0	0	1	PWM, phase and frequency correct	OCR1A	BOTTOM	BOTTOM
10	1	0	1	0	PWM, phase correct	ICR1	TOP	BOTTOM
11	1	0	1	1	PWM, phase correct	OCR1A	TOP	BOTTOM
12	1	1	0	0	CTC	ICR1	Immediate	MAX
13	1	1	0	1	(Reserved)	–	–	–
14	1	1	1	0	Fast PWM	ICR1	BOTTOM	TOP
15	1	1	1	1	Fast PWM	OCR1A	BOTTOM	TOP

Note: 1. The CTC1 and PWM11:0 bit definition names are obsolete. Use the WGM12:0 definitions. However, the functionality and location of these bits are compatible with previous versions of the timer.

Figure 5. TCCR1B register and its WGM12 and CS12 bits

The variables digit, paused, saved_tcmt1, print_flag and saved_state are volatile because they are modified within interrupt service routines and they are also modified outside of interrupt service routines.

Timing

Arduino Uno's clock is 16 MHz. With a Timer1 prescaler of 256:

$$f_{timer} = \frac{16,000,000}{256} = 62,500 \text{ Hz}$$

Each timer tick is:

$$T_{tick} = \frac{1}{62,500} = 16\mu s$$

For a 0.5 second interval:

$$0.5s \div 16\mu s = 31,250 \text{ ticks}$$

Since CTC mode counts from 0 to OCR1A, the compare value must be:

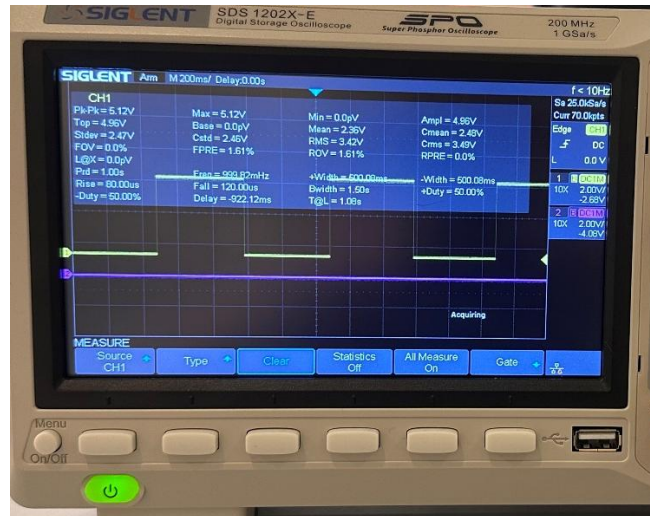
$$OCR1A = 31,250 - 1 = 31,249$$

Thus Timer1 will be generating an interrupt of compare for every 0.5 seconds. The oscilloscope readout showed approximately 500 ms pulse width and about 999.84 mHz frequency, which is approximately 1 Hz for a complete HIGH-LOW toggle cycle on the evidence pin. This supports that the display updates every 0.5 seconds. (See Figure 6)

The pause does not reset the digit. But Timer1 still keeps running when it's paused. That is, the digit shown will resume from the same number, but the saved TCNT1 values in the Serial Monitor at pause and resume will not be the same, as the button may be pushed at different times within the 0.5 second timer cycle. (See Figure 8)

Evidence explanation

There are two important signals on the oscilloscope evidence. One channel displays the timing evidence from PD7, which is toggled each time the digit changes. The measured width is approximately 500ms, which is the same as the calculated 0.5s interval. This is to show the counter is not using the software delay but Timer1 compare interrupts.

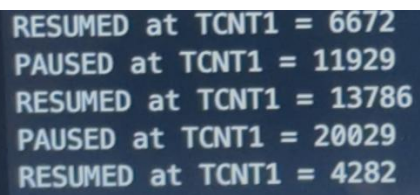


(if it's not visible here visit the [link](#) for the image please)

The second channel of the oscilloscope displays the pause state of a PD6. When PD6 is HIGH, the system is paused, and the digit update waveform stops changing. If PD6 goes LOW again, the counter is restarted. This indicates the external interrupt has been shown to toggle the pause state.



The evidence is seen in the Serial Monitor that messages like PAUSED at TCNT1 = ... and RESUMED at TCNT1 = appear. The values are not the same because the timer has continued to advance during the pause. This is expected. What is important is that the displayed digit does not start from 0, but from where it was stopped.



A screenshot of a serial monitor window with a dark blue background and white text. The text displays a sequence of five lines, each indicating a state change (RESUMED or PAUSED) followed by the current TCNT1 register value. The values are 6672, 11929, 13786, 20029, and 4282, showing an increasing trend with some irregularity between resume and pause events.

```
RESUMED at TCNT1 = 6672  
PAUSED at TCNT1 = 11929  
RESUMED at TCNT1 = 13786  
PAUSED at TCNT1 = 20029  
RESUMED at TCNT1 = 4282
```

Figure 8. Serial Monitor showing TCNT1 value at pause/resume

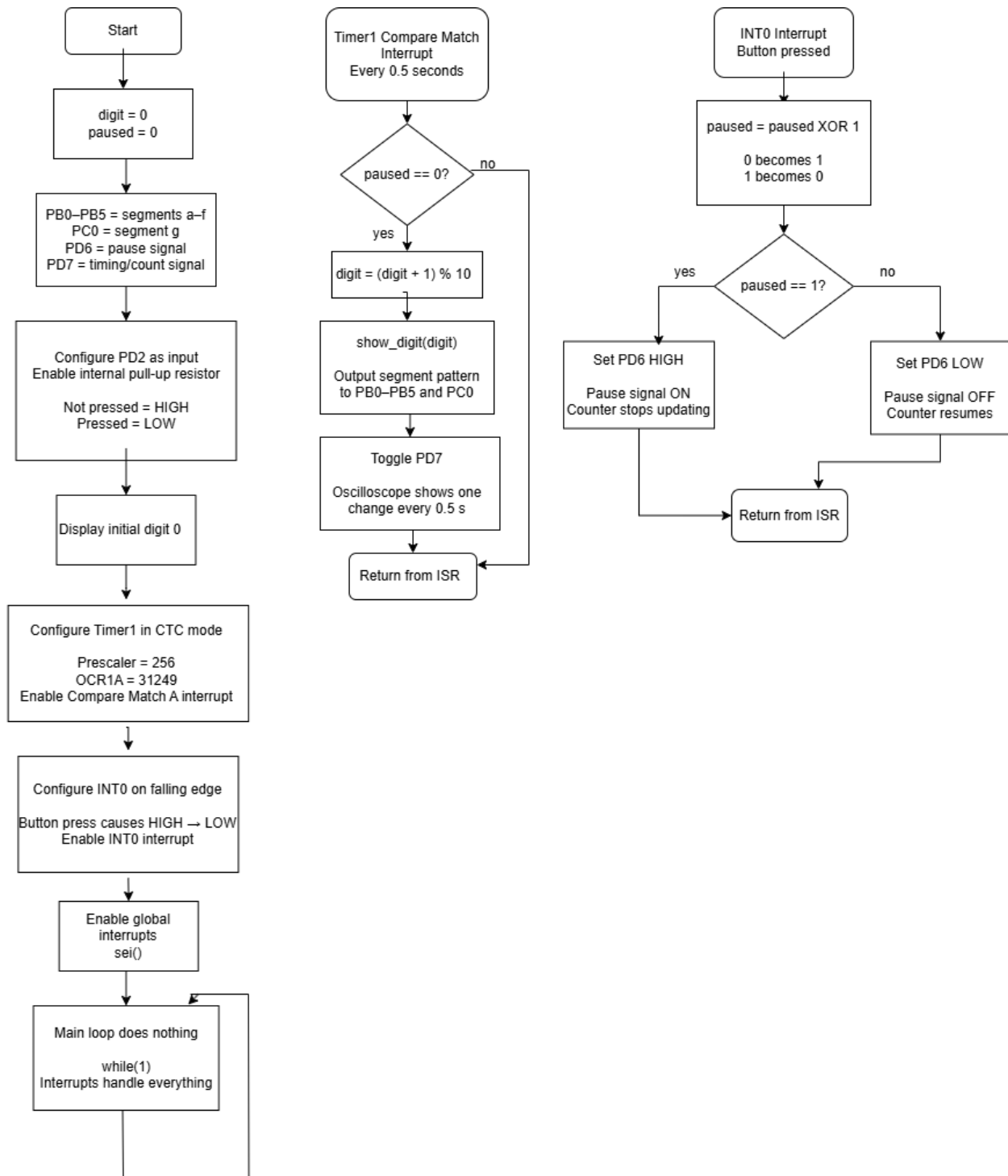


Figure 9. Flowchart for Task 6

Task 7: Your goal is to use two ATmega328P Arduinos to demonstrate basic SPI communication at the register level. One Arduino must act as the Master and the other as the Slave. The Master must send a single decimal value (0–255) once every second through SPI, and the Slave must correctly receive that value and display it on the Serial Monitor. You must configure all SPI settings using the SPCR, SPSR, SPDR, DDRB, and PORTB registers only.

The Master must transmit a sequence of three specific decimal values every second in the following order: 85 → 170 → 255 repeat

Evidence how can you show what is happening with SS, SCK, and MOSI

Draw a timing diagram for one of the decimal values showing how the digital signals (SS, SCLK, MOSI, MISO) change over time.

GitHub Solution: <https://github.com/catponyz/Microprocessor-Systems/tree/main/task%207>

Demo Video: <https://youtu.be/fSw1YmivDls>

Hardware Setup

Two Arduino Uno boards were used, which has ATmega328P microcontrollers. Both Arduinos were programmed using SPI, one as the slave and the other as the master. The SPI pins were connected directly between the boards: Master MOSI PB3 / D11; Slave MOSI PB3 / D11, Master MISO PB4 / D12; Slave MISO PB4 / D12, Master SCK PB5 / D13; Slave SCK PB5 / D13, Master SS PB2 / D10; Slave SS PB2 / D10. Both the grounds of Arduinos are also tied together, since they need to have the same reference voltage to communicate.

The oscilloscope probes were attached to the SPI lines to observe the behavior of SS, SCK, MOSI and MISO signals during transmission.

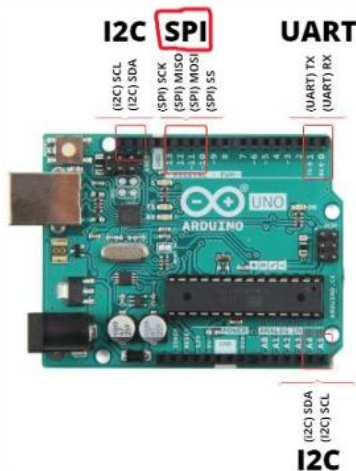


Figure 1. SPI pins on Arduino

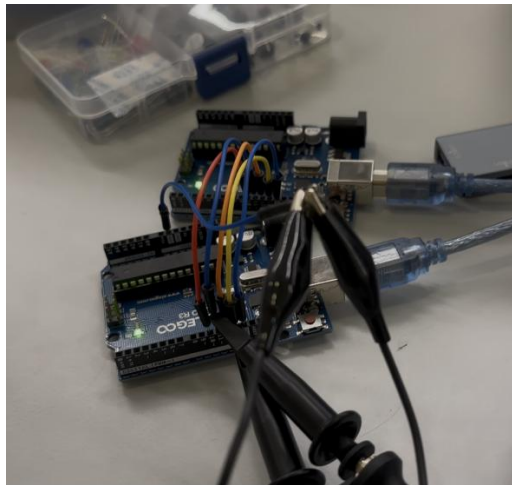


Figure 2. Hardware configuration for Task 7

Description

This task shows how to make a register-level SPI communication between two ATmega328P microcontrollers. SPI is in general a synchronous serial communication protocol. The data that is to be transferred is controlled by a clock signal. On SCK, the Master is generating the clock, the Slave is selected by SS, and data is sent on MOSI. Meanwhile, the Slave can transmit data back via MISO.

One of the Arduinos is programmed as Master and the other as Slave in the code. The Master is broadcasting three decimal numbers: 85, 170, and 255. After sending 255, the sequence repeats. The delay between each transmission is 1 second, thus the Slave receives and prints 1 value per second.

The Master writes into the SPI Data Register (SPDR) to send data. When writing to SPDR, the SPI transfer automatically begins. The Master then waits for the SPIF flag in SPSR to be set to 1 (transfer complete). Each byte sent, the Master pulls SS LOW to select the Slave. The byte is then transmitted and SS is immediately pulled HIGH again to terminate the transmission.

Until the Slave's SPIF flag is 1, the Slave waits. In this case one whole byte is received. Then, it reads the received value from SPDR.

Here CPOL and CPHA are set to 0 in SPCR and the SPI mode is the default mode. This implies that the device is set to have SPI Mode 0:

$$CPOL = 0, CPHA = 0$$

If CPOL = 0, then SCK will be LOW during idle. If CPHA = 0 then data is sampled on the leading edge of the clock. The idle clock is LOW so the leading edge is the rising edge. Thus, the Slave must read MOSI on every rising edge of SCK, and the data is changed on the falling edge of SCK.

18.5.1 SPCR – SPI Control Register

Bit	7	6	5	4	3	2	1	0	
0x2C (0x4C)	SPIE	SPE	DORD	MSTR	CPOL	CPHA	SPR1	SPR0	SPCR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Table 18-3. CPOL Functionality

CPOL	Leading Edge	Trailing Edge
0	Rising	Falling
1	Falling	Rising

Table 18-4. CPHA Functionality

CPHA	Leading Edge	Trailing Edge
0	Sample	Setup
1	Setup	Sample

Figure 3. SPCR register showing SPE, MSTR, CPOL, CPHA

Register / datasheet justification

The direction of the SPI pins is set using DDRB. On the Master, PB3 / MOSI, PB5 / SCK and PB2 / SS are outputs, as the Master transmits data, provides the clock control signal and controls Slave Select. Input: PB4 / MISO would be the pin that the Master receives data from.

PB4 / MISO is an output, while PB3 / MOSI, PB5 / SCK and PB2 / SS are inputs on the Slave. This is because the Slave receives data from MOSI, the clock from SCK and is selected by SS. MISO is output since it is the return data line from Slave to Master.

On the Master, SS is controlled by PORTB. If PORTB2 is set HIGH, the Slave is inactive. Clearing PORTB2 sets SS LOW (selects the Slave for communication) That's why the oscilloscope only displays the SS going LOW while SPI is active.

The SPI Control Register is called SPCR. In the Master, SPI is enabled by using the SPR1 to set the SPI clock divider and MSTR to select Master mode. Since SPR1 = 1 and SPR0 = 0, the SPI clock is:

$$f_{SCK} = \frac{16MHz}{64} = 250kHz$$

Thus, one SCK period is (See any Figure 6-9):

$$T = \frac{1}{250000} = 4\mu s$$

The transfer of one SPI byte would require approximately:

$$8 \times 4\mu s = 32\mu s$$

Table 18-5. Relationship Between SCK and the Oscillator Frequency

SPI2X	SPR1	SPR0	SCK Frequency
0	0	0	$f_{osc}/4$
0	0	1	$f_{osc}/16$
0	1	0	$f_{osc}/64$
0	1	1	$f_{osc}/128$
1	0	0	$f_{osc}/2$
1	0	1	$f_{osc}/8$
1	1	0	$f_{osc}/32$
1	1	1	$f_{osc}/64$

Figure 4. SPCR register showing SPR1, SPR0

Only SPE is set on the Slave. The Arduino is set as Slave, since MSTR is not set. The SPI clock is provided by the Master, thus Slave does not specify the clock speed.

The SPI Status Register is called SPSR. Once a complete SPI byte transfer is done, the SPIF bit is set to 1. In the Master code it is checked after writing to SPDR. In the Slave code, it is checked prior to read of received byte.

18.5.2 SPSR – SPI Status Register

Bit	7	6	5	4	3	2	1	0	
0x2D (0x4D)	SPIF	WCOL	–	–	–	–	–	SPI2X	SPSR
Read/Write	R	R	R	R	R	R	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Bit 7 – SPIF: SPI Interrupt Flag**

When a serial transfer is complete, the SPIF Flag is set. An interrupt is generated if SPIE in SPCR is set and global interrupts are enabled. If $\overline{SS}$ is an input and is driven low when the SPI is in master mode, this will also set the SPIF flag. SPIF is cleared by hardware when executing the corresponding interrupt handling vector. Alternatively, the SPIF bit is cleared by first reading the SPI status register with SPIF set, then accessing the SPI data register (SPDR).

Figure 5. SPSR register showing SPIF bit

SPDR stands for the SPI Data Register. Writing to SPDR will begin the transfer on the Master. On the Slave, when reading SPDR, the received byte is returned. The bits of a decimal are sent as full 8-bit transmissions, as SPI expects 8 bits per byte.

For the value 85:

$$85_{10} = 01010101_2$$

For the value 170:

$$170_{10} = 10101010_2$$

For the value 255:

$$255_{10} = 11111111_2$$

These binary values are what make up the MOSI waveform. With 85 and 170, the MOSI signal will be LOW and HIGH during the clock pulses. The MOSI bus remains HIGH for all 8 bits for 255. (See Figures 7-9)

Evidence explanation

The oscilloscope captures support the SPI behavior. The SCK waveform shows the clock pulses generated by the Master. Because SPI is synchronous, the Slave reads the data on MOSI in sync with these clock pulses.

The SS waveform indicates that SS is only LOW in the byte transfer. This shows that the Master is choosing Slave before sending data, and releasing Slave when data is sent. If SS is HIGH, then the Slave is not active.



Figure 6. Oscilloscope capture of SS and SCK during one SPI transfer

The MOSI waveform displays the actual bits that are being sent. For instance, if the number 85 is transmitted, the waveform will be 01010101. The waveform that is transmitted for 170 is 10101010. If the number 255 is sent, then MOSI will stay HIGH throughout the eight clock pulses. This verifies that the Master is sending the correct decimal sequence.

The oscilloscope captures below show the MOSI patterns for each transmitted value:

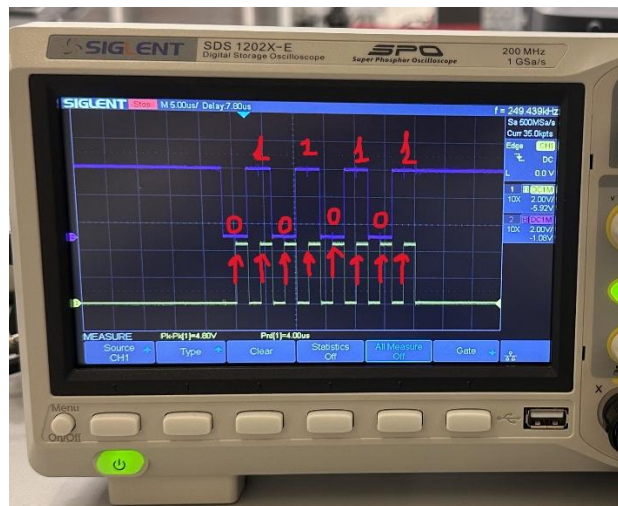


Figure 7. Oscilloscope capture of SCK and MOSI for value 85

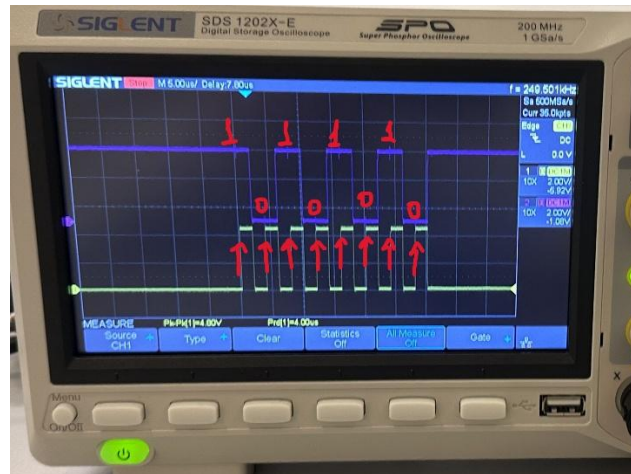


Figure 8. Oscilloscope capture of SCK and MOSI for value 170

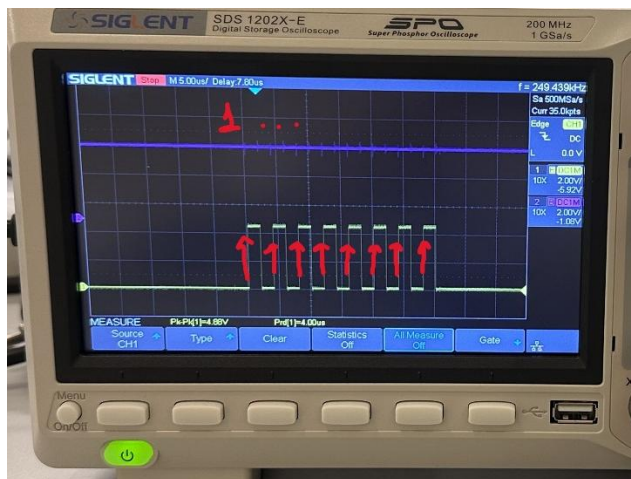


Figure 9. Oscilloscope capture of SCK and MOSI for value 255

Activity is indicated in the MISO capture during clock pulses; in this task, no use is made of the data received by the Master. Thus, MISO is not important for the decimal output required. The important ones are that when SCK is active, MOSI should be carrying the correct byte with LOW SS.

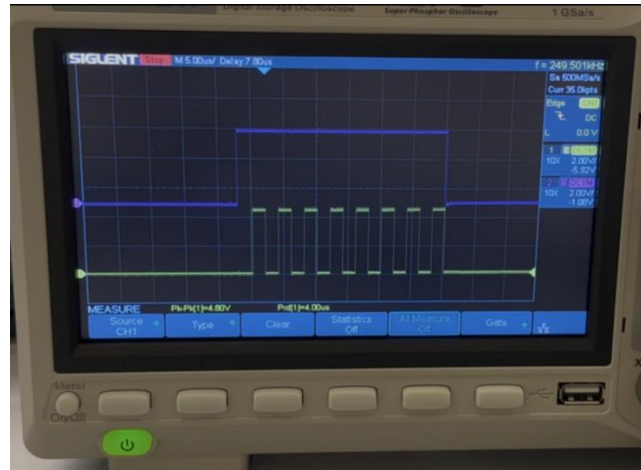


Figure 10. Oscilloscope capture of SCK and MISO; MISO is not used for the required decimal output.

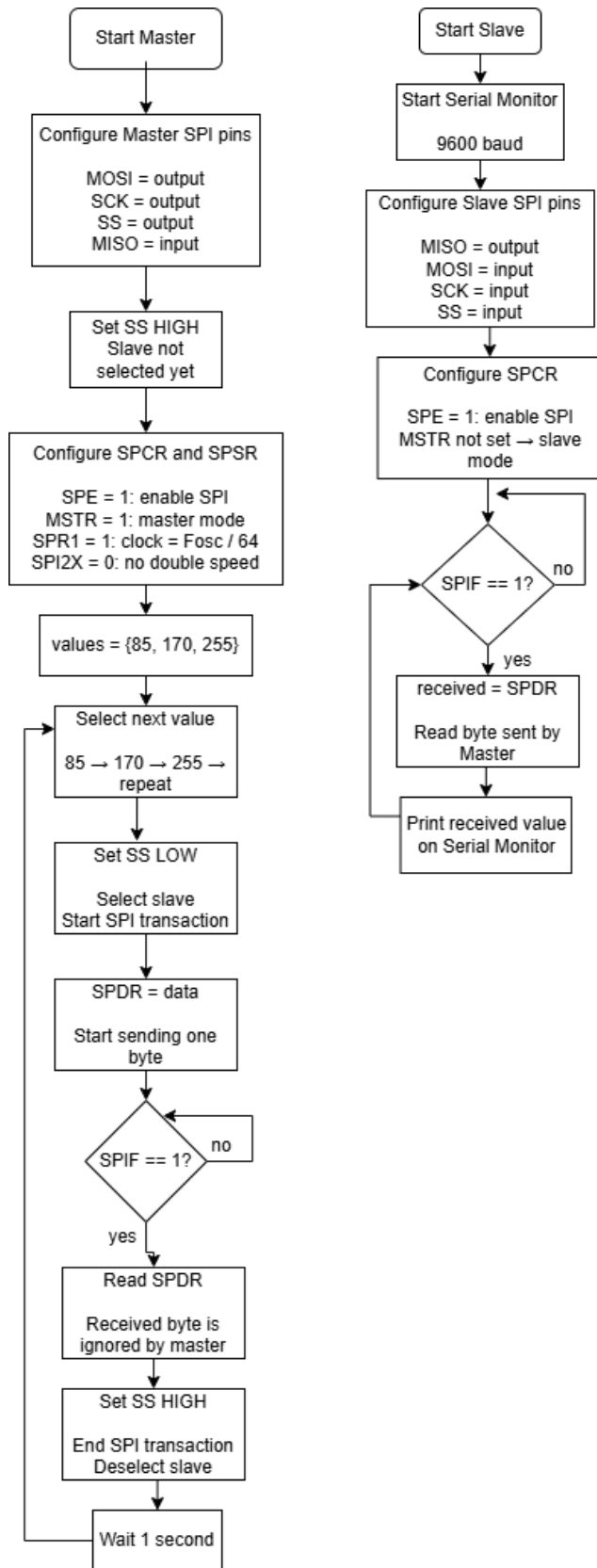


Figure 11. Flowchart for Task 7

TASK 8: Set up two Arduino boards communicating via I2C (one master, one slave), each with a push button and LED; pressing Button A should send a message to Board B to light its LED, and pressing Button B should send its state back to Board A to light its LED. Explain your message protocol design in your report. Then, explore how clock speed affects the I2C signals by modifying the TWI clock registers (TWBR and prescaler bits in TWSR) to test at least two different speeds. Use an oscilloscope to observe the SDA and SCL waveforms at each speed and document the timing characteristics (bit period, rise/fall times, signal quality, and any ringing or noise). Discuss how speed changes affect the observed waveforms, any timing violations or signal degradation you encounter, and the practical limits of increasing clock speed on your specific hardware setup
→ include oscilloscope screenshots and explain what constraints (capacitance, trace length, pull-up resistors, or device slew rates) are likely responsible for any limitations you observe.

GitHub Solution: <https://github.com/catponyz/Microprocessor-Systems/tree/main/task%208>

Demo Video: https://youtu.be/ayUupy_fTOI

Hardware Setup

Two Arduino Uno were used one as I2C Master and the other as I2C Slave. SDA was linked between A4/PC4 on both boards and SCL was linked between A5/PC5 on both boards. The two grounds of both Arduinos were shared. Push-button with internal pull-up on each board was connected to the PD2, and the onboard LED on the PB5/D13 was used as the output LED.

The external pull-up resistors on SDA and SCL were tested by connecting the resistors to 5 V. The cases that were tested are 330 Ω , 2 k Ω , and no external resistor. To compare the quality of the oscilloscope output when connected to SDA and SCL with various clock speeds and pull-up conditions, the oscilloscope was connected to SDA and SCL.

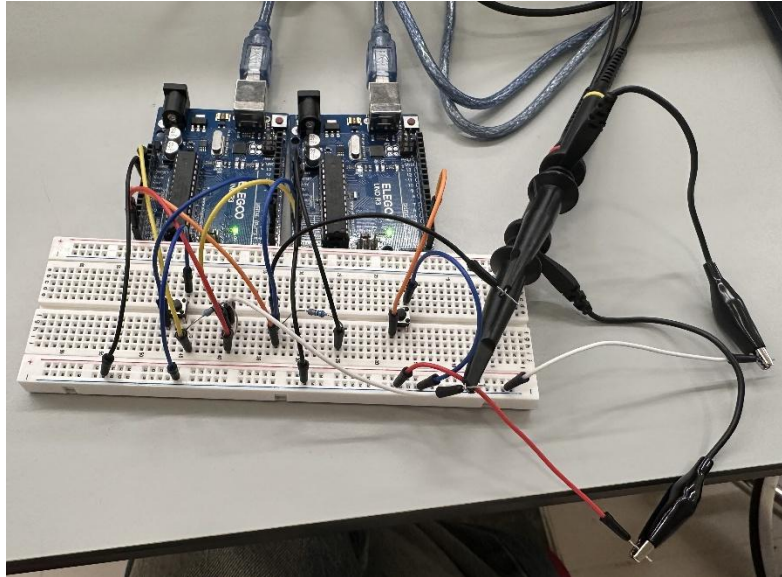


Figure 1. Hardware configuration for Task 8

Description

I2C, also called TWI on the ATmega328P, is a two-wire serial communication protocol. It has SDA as data and SCL as clock. Both are open-drain/open-collector type, with devices tending to pull the line LOW and the HIGH state being established by pull-up resistors. This is why the pull-up strength has a significant impact on the waveform shape particularly at higher speeds.

The Master reads its button and sends a message to the Slave in this task. The Slave reads its own button, and communicates its state back to the Master when asked. The protocol of the message is:

0x00 = no button pressed / LED off

0x01 = Master button pressed, turn Slave LED on

0x02 = Slave button pressed, turn Master LED on

First the Master sends either 0x01 or 0x00 to the Slave, depending upon button state. It then sends a request for 1 byte to the Slave. The Master will turn its LED on if it receives the 0x02 byte, otherwise it will turn it off.

The Slave checks TWI status codes to determine what occurred on the bus. When Slave receives data from the Master, the Slave reads TWDR, and LED turns on only if the data received is 0x01. When the Master asks for data, the Slave will load the current state of its buttons into TWDR (0x02 or 0x00).

Register / datasheet justification

TWBR sets the I2C bit rate together with the prescaler bits in TWSR. The formula is:

$$SCL = \frac{F_{CPU}}{16 + 2 \cdot TWBR \cdot prescaler}$$

The prescaler in the code is set to 1 with TWSR = 0x00. The value for TWBR is 72, which gives about 100 kHz for standard mode. TWBR = 12 will provide an approximate frequency of ~400 kHz for fast mode.

21.9.1 TWBR – TWI Bit Rate Register

Bit	7	6	5	4	3	2	1	0	
(0xB8)	TWBR7	TWBR6	TWBR5	TWBR4	TWBR3	TWBR2	TWBR1	TWBR0	TWBR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 2. TWBR register

Rearranged for finding TWBR:

$$TWBR = \frac{\left(\frac{F_{CPU}}{SCL}\right) - 16}{2 \cdot prescaler}$$

Since $F_{CPU} = 16MHz$ and $prescaler = 1$:

For 100 kHz:

$$TWBR = \frac{160 - 16}{2} = 72$$

For 400 kHz:

$$TWBR = \frac{40 - 16}{2} = 12$$

21.9.3 TWSR – TWI Status Register

Bit	7	6	5	4	3	2	1	0	
(0xB9)	TWS7	TWS6	TWS5	TWS4	TWS3	–	TWPS1	TWPS0	TWSR
Read/Write	R	R	R	R	R	R	R/W	R/W	
Initial Value	1	1	1	1	1	0	0	0	

Table 21-8. TWI Bit Rate Prescaler

TWPS1	TWPS0	Prescaler Value
0	0	1
0	1	4
1	0	16
1	1	64

Figure 3. TWSR register showing TWPS1/TWPS0 prescaler bits

The TWCR is the TWI Control Register. TWEN turns on the TWI hardware. TWINT is set during a TWI operation when the operation is completed, it must be written as 1 to clear it and to continue. A condition START is created by TWSTA, a condition STOP is created by TWSTO and a condition ACK response is enabled on the Slave by TWEA.

21.9.2 TWCR – TWI Control Register

Bit	7	6	5	4	3	2	1	0	
(0xBC)	TWINT	TWEA	TWSTA	TWSTO	TWWC	TWEN	–	TWIE	TWCR
Read/Write	R/W	R/W	R/W	R/W	R	R/W	R	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Figure 4. TWCR register showing TWINT, TWSTA, TWSTO, TWEN, TWEA

The TWI Data Register is called TWDR. When on the Master, it stores the address or data byte being sent. For the Slave it stores the received byte or the byte that will be sent back to the Master.

TWAR stores the Slave address. In the Slave code, $TWAR = (SLAVE_ADDR \ll 1)$ because bit 0 is reserved for general call recognition, so the 7-bit address must be shifted left.

TWSR also has status codes. The slave masks it with 0b11111000 as the lower bits are used to carry prescaler information. The states used were 0x60 (own address + write received), 0x80 (data received) and 0xA8 (own address + read received).

Table 21-5. Status Codes for Slave Receiver Mode

Status Code (TWSR) Prescaler Bits are 0	Status of the 2-wire Serial Bus and 2-wire Serial Interface Hardware	Application Software Response					Next Action Taken by TWI Hardware
		To/from TWDR	To TWCR				
			STA	STO	TWINT	TWEA	
0x60	Own SLA+W has been received; ACK has been returned	No TWDR action or	X	0	1	0	Data byte will be received and NOT ACK will be returned
		No TWDR action	X	0	1	1	Data byte will be received and ACK will be returned
0x68	Arbitration lost in SLA+R/W as Master; own SLA+W has been received; ACK has been returned	No TWDR action or	X	0	1	0	Data byte will be received and NOT ACK will be returned
		No TWDR action	X	0	1	1	Data byte will be received and ACK will be returned
0x70	General call address has been received; ACK has been returned	No TWDR action or	X	0	1	0	Data byte will be received and NOT ACK will be returned
		No TWDR action	X	0	1	1	Data byte will be received and ACK will be returned
0x78	Arbitration lost in SLA+R/W as Master; General call address has been received; ACK has been returned	No TWDR action or	X	0	1	0	Data byte will be received and NOT ACK will be returned
		No TWDR action	X	0	1	1	Data byte will be received and ACK will be returned
0x80	Previously addressed with own SLA+W; data has been received; ACK has been returned	Read data byte or	X	0	1	0	Data byte will be received and NOT ACK will be returned
		Read data byte	X	0	1	1	Data byte will be received and ACK will be returned

Table 21-6. Status Codes for Slave Transmitter Mode

Status Code (TWSR) Prescaler Bits are 0	Status of the 2-wire Serial Bus and 2-wire Serial Interface Hardware	Application Software Response					Next Action Taken by TWI Hardware
		To/from TWDR	To TWCR				
			STA	STO	TWINT	TWEA	
0xA8	Own SLA+R has been received; ACK has been returned	Load data byte or	X	0	1	0	Last data byte will be transmitted and NOT ACK should be received Data byte will be transmitted and ACK should be received
		Load data byte	X	0	1	1	

Figure 5. Used Status codes

The LED is set as output by DDRB at PB5/D13. LED. DDRD clears PD2 as input for the button, and PORTD2 enables the internal pull-up. Weak internal pull-ups are enabled on SDA and SCL by PORTC4 and PORTC5, but external pull-ups were tested due to the weakness of internal pull-ups for clean high speed I2C.

Timing

At 100 kHz, the expected bit period is (See Figure 6):

$$T = \frac{1}{100000} = 10\mu s$$

At 400 kHz, the expected bit period is (See Figure 7):

$$T = \frac{1}{400000} = 2.5\mu s$$

When the pull ups were strong, the oscilloscope indicated that the bus could attain a value very close to the fast mode value. The pull-up resistance used was 330 Ω giving a waveform that was closest to square and the measured clock was approximately 400 kHz (See Figure 7). The 2 k Ω clock was still very close to 399 kHz but now the rising edge was slower and less sharp (See Figure 8). However, when no external resistor was placed in the circuit, the measured clock was about 342 kHz and the output was almost triangular in shape, indicating that the line was taking too long to rise (See Figure 9).

This is because SDA and SCL are not actively set HIGH. The rise time is a function of the pull-up resistor and bus capacitance. The longer the wires, the greater the breadboard capacitance, the greater the probe capacitance, and the weaker the pull-ups, the more slowly the line will rise. The shorter wires brought better signal due to their reduced extra capacitance and noise.

This can be estimated using:

$$\tau = R \times C$$

Where τ is the RC time constant, R is the pull-up resistance, and C is the total bus capacitance from wires, breadboard, Arduino pins, and oscilloscope probes. A smaller pull-up resistor gives a smaller τ , so the signal rises faster. This explains why the 330 Ω pull-up produced a sharper waveform than 2 k Ω or no external resistor.

Evidence explanation

The oscilloscope captures of SCL and SDA displays the I2C activity directly. SCL generates the clock pulses; SDA is used to change based on the address and data bits. The SDA trace also indicates that the data line is being set to HIGH or LOW while transmitting the data, corresponding to the message data that the two boards are communicating.

SDA and SCL were able to get to a valid HIGH level at 100 kHz because the bit period was longer, the wave form was more stable. The effect of the pull-up resistor was clearly seen at 400 kHz. The sharpest rising edges were obtained for the 330 Ω case; the 2 k Ω case, had slower rising edges; and the no-resistor case exhibited the worst wave form quality.



Figure 6. I2C waveform at 100 kHz with SDA and SCL

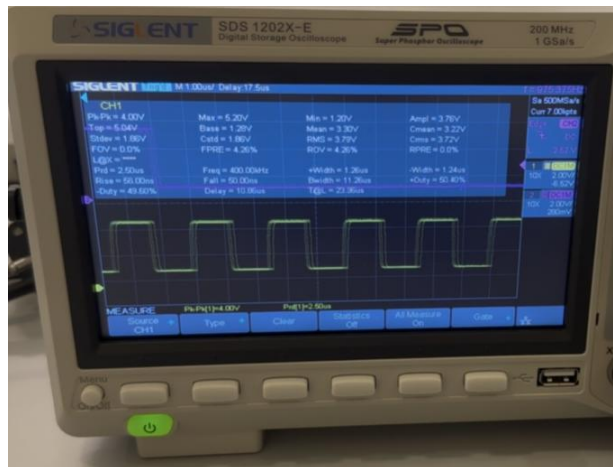


Figure 7. I2C waveform at 400 kHz with 330 Ω pull-ups

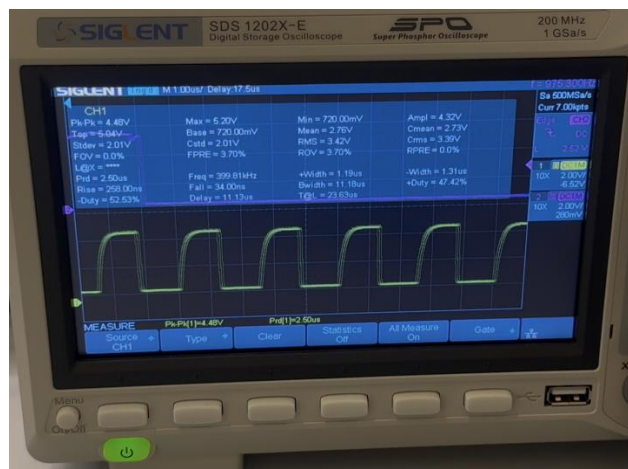


Figure 8. I2C waveform at 400 kHz with 2 kΩ pull-ups



Figure 9. I2C waveform at failed 400 kHz (342 kHz) with no external pull-ups

The outcome reveals the rationale behind the use of pull-up resistors in I2C. The line needs to return to HIGH faster than it is pulled back, otherwise it will not get pulled back to HIGH at the next clock edge. This results in waveforms that are rounded or formed into a triangle and finally can cause timing violations or incorrect data reading. The following factors were the most prominent factors limiting the practical speed in this configuration: pull-up resistance, wire length, breadboard capacitance and physical quality of connections.

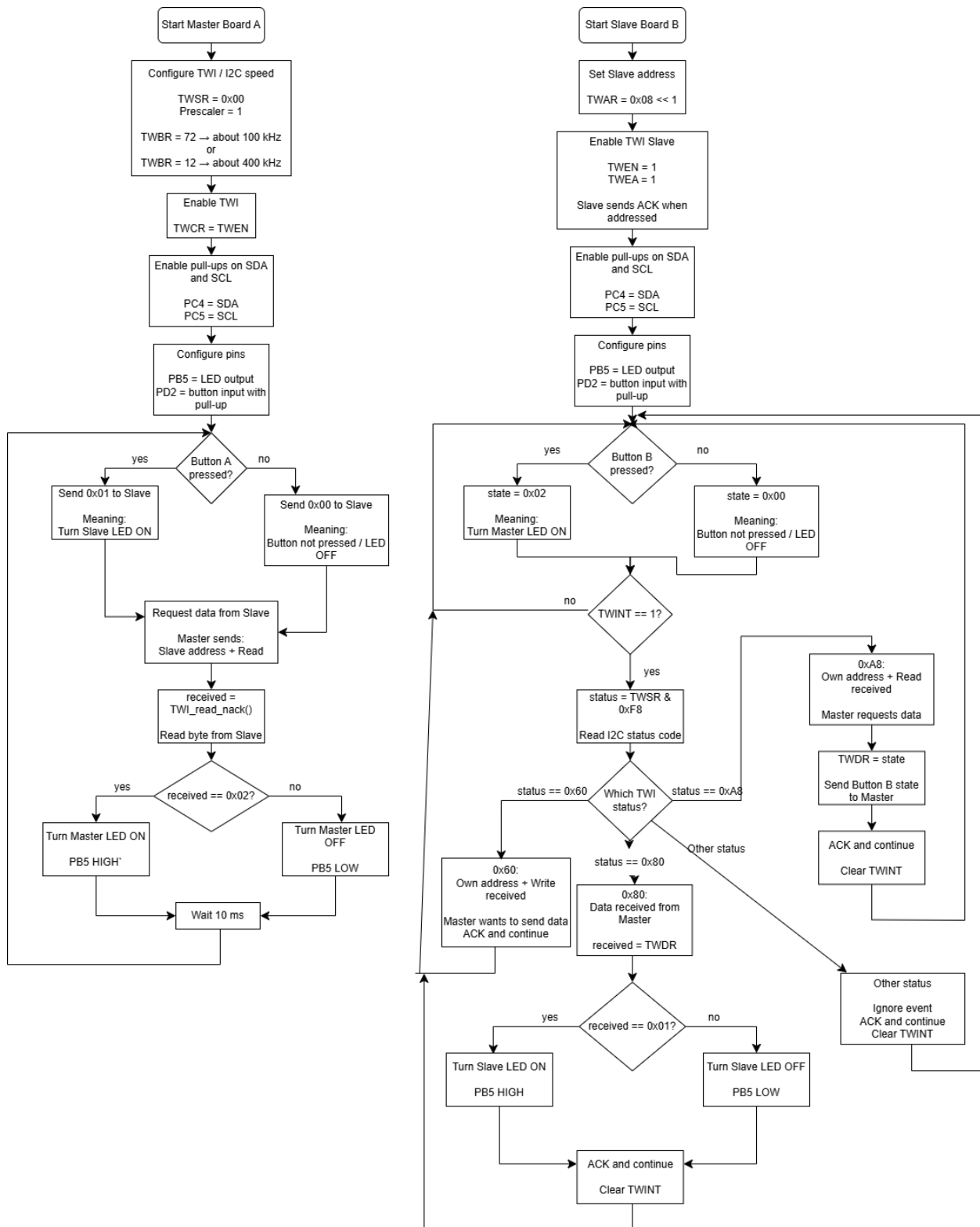


Figure 10. Flowchart for Task 8

References

- [1] Circuito.io. "Arduino Uno Pinout Diagram." Accessed for Arduino Uno pin mapping. <https://www.circuito.io/blog/arduino-uno-pinout/>
- [2] diagrams.net. "diagrams.net." Accessed for preparing flowcharts. <https://app.diagrams.net/>
- [3] Microchip Technology Inc. *ATmega328P Datasheet*. Used for register descriptions and bit-field definitions. https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf
- [4] Microchip Technology Inc. *AVR Instruction Set Manual*. Used for AVR instruction behavior and cycle counts. <https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf>